卒業論文　2025 年度

Double-Cut-And-Join 操作による ゲノム間距離に関する 線形時間アルゴリズムの再実装

平田 智也
（学籍番号: 62216580）

指導教員　小田 芳彰 准教授

2026 年 1 月

慶應義塾大学理工学部数理科学科 4 年
小田研究室

目次

論文要旨 ・・・

論文要旨

生物学において進化の過程を論じる際，突然変異を経て変化する世代間の影響を考慮することは不可欠である．また，生物学的背景から発生しやすい変異とそうでない変異を区別し，前者を中心に遺伝的な距離を定義し，考察することは自然なアプローチと言える．

Hannenhalli および Pevzner (1995)[2] は，染色体に作用する操作を「転座」「融合」「分裂」「反転」の 4 種類に制限し，遺伝的距離 d_{HP} を定義した．しかしながら，この手法および提案されたアルゴリズムには，計算上の制約や複雑性に関する課題が存在した．

Bergeron ら (2009)[1] は，これらの問題を解決すべく，上記 4 種類の操作を一般化した「Double-Cut-And-Join（以下，DCJ）という単一の操作を導入し，新たなアルゴリズムを提案した．DCJ 操作に基づく距離を d_{DCJ} と呼ぶ．

Bergeron らの手法は，染色体内の特定パターンを検出することで線形時間での操作手順出力を可能にしているが，その実装は極めて高度であり，直感的な理解が困難な側面がある．本論文では，DCJ 理論における Bergeron らと同様の前提条件の下，C 言語を用いたより簡潔な線形時間アルゴリズムの実装について述べる．本稿の前半では d_{DCJ} の算出に特化したアルゴリズムを，後半では操作手順も出力する，より汎用性の高いシンプルな実装を紹介する．

実装においては，一部の着想を Bergeron ら (2009) の研究に依拠しつつも，主要なデータ構造および処理ロジックの大部分は独自に構築したものである．

第 1 章　導入

1.1　背景

計算分子生物学において遺伝的距離をモデル化する手法には多くの変遷があるが，その基本的な枠組みは共通している．単純性の原理（Principle of Parsimony）のもとでは，可能な限り小さい回数の変化を知ることが重要になる．つまり，最終的な目標は，二つのゲノム間の最小変異回数を算出し，それらを比較・評価することになる．

1.2　ゲノムおよび染色体の表現方法

比較対象となる二つのゲノムを，それぞれ *source* および *target* と呼ぶ．各ゲノムは染色体（chromosome）の集合であり，各染色体は遺伝子（gene）の並びである．

各遺伝子は正負の符号を持つ整数値で識別する．向きの基準を *target* ゲノムに置き，絶対値の対応づけは任意であるため， *target* が昇べきの順となるようにつける．すなわち，遺伝子数 n のゲノムにおいて，*target* が単一の線形染色体である場合，その表現は常に $\langle 1,\ 2,\ \ldots,\ n \rangle$ と一意に定まる．線形染色体と環状染色体を区別するため，本論文では以下の表記を用いる．

- **線形染色体**: $\langle \ldots \rangle$ で囲む.
- **環状染色体**: $(\ldots)$ で囲む.

注意 1.1　環状染色体 $(5,\ 6,\ 7)$ は，要素を巡回させた $(6,\ 7,\ 5)$ や $(7,\ 5,\ 6)$ と同一である．

1.3　端点（extremity）と隣接（adjacency）

アルゴリズムの実装においては，遺伝子間の「接続関係」を直接扱う方が効率的である．遺伝子 g に対し，二つの **端点（extremity）**を，右側の端点 g および左側の端点 $-g$ と定義する．二つの遺伝子の接続は，これら端点の 2 要素の集合として表現され，これを **隣接（adjacency）** と呼ぶ.

また，線形染色体の末端については，特殊なシンボル o を用いて**テロメア**

(**Telomere**)[1] との接続{o, x} として扱う．この定義により，ゲノムは単なる隣接関係の集合として記述でき，線形・環状の区別や染色体数を意識せずに統合的な操作が可能となる．

なお，実装上の細かな差異について付記すると，**adjacency** は 2 つの **extremity** から構成されるため，いずれのアルゴリズムでも基礎的なデータとして両者が必要となるが，主な操作対象には違いがある．第 3 章のアルゴリズム 1（距離計算）では隣接グラフの構造把握が主目的であるため **adjacency** の走査が行われるが，第 4 章のアルゴリズム 2（操作手順出力）では接続替えの操作を行うため **extremity** を直接操作する．

本研究での実装において，これらの概念は以下の構造体として定義される．

Listing 1.1　extremity 構造体の定義

```
typedef struct extremity
{
    int id; // 遺伝子ID (1 ...)
    int sign; // 向き (1 or -1)
    struct extremity *reality; // ゲノムAにおける接続先
    struct extremity *desire; // ゲノムBにおける接続先
} extremity;
```

Listing 1.2　adjacency 構造体の定義

```
typedef struct adjacency
{
    extremity *left; // 隣接の左端点
    extremity *right; // 隣接の右端点
    struct adjacency *left_to; // グラフ上の左側エッジ
    struct adjacency *right_to; // グラフ上の右側エッジ
    int visited; // 探索用フラグ
} adjacency;
```

1） 生物学において染色体の末端を保護する構造体に由来する名称である．

第 2 章　理論的背景

2.1　d_{DCJ} に関する等式

Bergeron ら (2006)[3] は，DCJ 操作を用いたゲノム距離の計算において，以下の重要な定義および定理を示している．

定義 2.1（Double-Cut-And-Join）　ゲノムの隣接関係の集合に対する操作であり，二つの隣接 $\{u, v\}$ と $\{x, y\}$ を切断し，新たに $\{u, x\}$ と $\{v, y\}$, または $\{u, y\}$ と $\{v, x\}$ として再結合する操作を指す．テロメア o を含む隣接に対しても同様に定義される．

また，$\{o, o\}$ との DCJ も認めることで，単なる隣接の切断も DCJ に含めるものとする．

例 2.2　以下のゲノム A, B を考える （$N = 5$）．

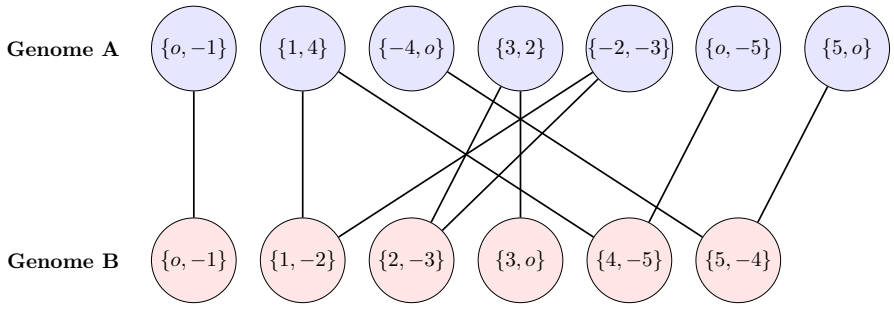
```
A: < 1 -4 > ( 3 -2 ) < 5 >
B: < 1 2 3 > ( 4 5 )
```

これらのゲノムは， 以下のような隣接の集合としても表現できる．

$$A = \{\{o, -1\}, \{1, 4\}, \{-4, o\}, \{3, 2\}, \{-2, -3\}, \{o, -5\}, \{5, o\}\}$$

$$B = \{\{o, -1\}, \{1, -2\}, \{2, -3\}, \{3, o\}, \{4, -5\}, \{5, -4\}\}$$

また， これらに基づく隣接グラフ（定義 2.3 参照）の成分は以下のようになる（円形のノードは， 青色がゲノム A の隣接， 赤色がゲノム B の隣接を表す）．



このとき， 以下の 4 回の DCJ 操作により A は B へと変換される．

```
DCJ(G): { 1, 4 } { -2, -3 } -> { 1, -2 } { 4, -3 }
DCJ(G): { 2, 3 } { -3, 4 } -> { 2, -3 } { 3, 4 }
DCJ(G): { -4, o } { 5, o } -> { -4, 5 } { o, o }
```

```
DCJ(G): { 4, 3 } { -5, o } -> { 4, -5 } { 3, o }
```

定義 2.3（隣接グラフ $AG(A,B)$） 2 つのゲノム A, B において，それぞれの隣接関係およびテロメアを頂点集合とし，共通の遺伝子端を持つ頂点間にエッジを引いたグラフを隣接グラフと呼ぶ．

ただし，共通の要素がテロメア o である場合はエッジを引かないものとする．また，2 つの頂点間で 2 つの遺伝子端が共通する場合，それぞれの端点に対応する計 2 本のエッジを引くこととする．したがって，隣接グラフは多重グラフとなり得る．

注意 2.4 それぞれの頂点の次数は高々 2 であるため，隣接グラフはサイクル（Cycle）とパス（Path）のみからなる．

補題 2.5（成分数の変化） 次数が高々 2 のグラフへの 1 回の DCJ 操作により，成分（サイクルまたはパス）の総数は最大で 1 つ増減する．

証明 DCJ 操作は高々 2 つの頂点（あるいは 1 つのエッジ）に作用する局所的な操作である．異なる成分に属する頂点間で操作を行う場合，2 つの成分が連結され 1 つの成分となるため，成分数は 1 減少する．逆に，同一成分に属する頂点間で操作を行う場合，その成分が 2 つに分割されるか（成分数は 1 増加），あるいは成分の構造が変化するのみで分割されない（成分数は不変）．したがって，1 回の操作による成分数の変化量は ± 1 または 0 であり，最大で 1 つしか増減しない． □

補題 2.6（奇数長パス数の変化） 隣接グラフにおいて，1 回の DCJ 操作は奇数長パスの数 I を 2 増加，変化なし，あるいは 2 減少させる．

証明 DCJ 操作がパス（線形成分）に作用するケースについて，パス長の偶奇性（辺の数が奇数か否か）の変化を考察する．例えば，2 つの奇数長パスが操作によって 1 つの偶数長パスにまとめられる場合，奇数長パスの数 I は 2 減少する．対照的に，1 つの偶数長パスが操作によって 2 つの奇数長パスに切り分けられる場合，I は 2 増加する．一方，サイクル（環状成分）のみに関わる操作については，本隣接グラフが二部グラフでありサイクル長が常に偶数である性質上，奇数長パスの増減には寄与しない．以上のパターンを網羅すると，1 回の操作における I の変化量は $\{-2, 0, 2\}$ のいずれかに限定される． □

補題 2.7（ゲノムの一致条件） N 個の遺伝子集合からなるゲノム A, B が $A = B$ であるための必要十分条件は，次式が成り立つことである：

$$N = C + \frac{I}{2} \tag{2.1}$$

ここで，C は隣接グラフ $AG(A,B)$ におけるサイクルの数，I は奇数長パス（辺の数が奇数のパス）の数である．

証明　ゲノム A における隣接の数を a，テロメアの数を t とおくと，遺伝子の総数は $N = a + t/2$ と表される．

まず必要性 ($A = B \implies N = C + I/2$) を示す．$A$ と B が同一であれば，任意の隣接 $\{u, v\} \in A$ は B における同一の隣接と，共有する 2 つの遺伝子端を介して 2 重に結合されるため，長さ 2 のサイクルを形成する．したがって，サイクルの総数は $C = a$ となる．同様に，各テロメア $\{u\} \in A$ は B の対応するテロメアと単一のエッジで結ばれ，長さ 1 のパス（奇数長パス）となることから，$I = t$ が従う．ゆえに，$C + I/2 = a + t/2 = N$ が成立する．

次に十分性 ($N = C + I/2 \implies A = B$) を示す．隣接グラフにおける辺の総数に着目する．各遺伝子は 2 つの端点を持ち，これらがゲノム A および B の頂点間で共有されるため，グラフ全体のエッジ数は $2N$ 本となる．また，本グラフは二部グラフであることから奇数長サイクルは存在せず，頂点の次数が高々 2 であることから偶数長パスも構成され得ない．

仮定より $2N = 2C + I$ が成り立つが，これはグラフ内の全エッジがサイクルまたは奇数長パスのみによって構成されていることを意味する．ここで，サイクルの最小長は 2，奇数長パスの最小長は 1 である．エッジ総数の制約と等式 $2C + I = 2N$ を考慮すれば，C 個のサイクルはすべて長さ 2 であり，I 本のパスはすべて長さ 1 でなければならないことが導かれる．

長さ 2 のサイクルはテロメアを含まない隣接関係の一致を，長さ 1 のパスはテロメアの一致をそれぞれ意味する．以上より，全ての隣接関係およびテロメアが完全に一致するため，$A = B$ が示された．　□

補題 2.8（DCJ 距離の下限）　任意の 2 つのゲノム A, B について，DCJ 距離は以下を満たす：

$$d_{\mathrm{DCJ}}(A, B) \geq N - \left(C + \frac{I}{2}\right) \tag{2.2}$$

証明　補題 2.5 および補題 2.6 より，1 回の DCJ 操作でサイクル数 C と奇数長パス数 I が同時に変化することはない．したがって，量 $C + I/2$ の 1 操作あたりの増加量は最大でも 1 である（C が 1 増えるか，I が 2 増える場合）．目標状態では補題 2.7 より $N = C + I/2$ であるため，初期状態からの差分を埋めるために少なくとも右辺の回数の操作が必要となる．　□

定理 2.9（DCJ 距離公式）　2 つのゲノム A, B 間の最短の DCJ 操作回数（DCJ 距離）$d_{\mathrm{DCJ}}(A, B)$ は，次式で与えられる：

$$d_{\mathrm{DCJ}}(A, B) = N - \left(C + \frac{I}{2}\right) \tag{2.3}$$

この距離を達成する最短の並べ替え手順は，線形時間 $O(N)$ で計算可能である．

証明　補題 2.8 により下限が示されているため，この回数でソートを行う手順が存在することを示せばよい．まず，$A \neq B$ である限り，B に存在して A に存在しない隣接（2 つの端点の接続），あるいはテロメアとの接続が必ず存在する（A と B が一致せず，かつこれら全てが一致している状況は存在しない）．第 4 章で詳述する貪欲法アルゴリズムは，そのような不一致箇所を解消するように操作を行うことで，常に $C + I/2$ を 1 ずつ増加させる操作（C を 1 つ増加させる，あるいは I を 2 つ増加させる操作）を選択し続けることができる．したがって，下限値でのソートが可能であり，これが最短距離となる．　□

2.2　距離公式に関する考察

(2.3) 式は，d_{DCJ} を計算する上で，隣接グラフの初期状態さえ構築できれば，実際にグラフを物理的に変形させる手順を踏まずとも，グラフ内のコンポーネント（サイクルおよびパス）を走査し C, I を計数するだけで距離が確定することを示している．この知見に基づき，まずアルゴリズム 1 を提案する．

第 3 章　アルゴリズム 1：距離計算の実装

本アルゴリズムは，隣接グラフを構築し，そこに含まれるサイクルおよび奇数長パスを線形時間で探索するものである．

3.1　入力形式

本実装では，以下のフォーマットに従うテキストファイルを入力とする．1 行目に遺伝子数 N，2 行目にゲノム A，3 行目にゲノム B を記述する．

```
5
< 1 -4 >( 3 -2 )< 5 >
< 1 2 3 >( 4 5 )
```

ここで < ... > は線形染色体，(...) は環状染色体を表し，内部の数字は遺伝子 ID を示す．

3.2　データ構造

実装においては，以下の 3 つの要素を中核とする．

- **extremity**: 遺伝子の端点を表す構造体．ID と符号を保持する．
- **adjacency**: 2 つの extremity のペア．探索を効率化するため，各隣接が *source* と *toget* のどちらに属するか，および訪問済みかどうかのフラグを持つ．
- **Map**: 特定の端点がどの adjacency に含まれているかを定数時間で検索するための整数値配列．

これらのデータ構造は，以下の手順によって入力から構築される．

Algorithm 1: 入力からのデータ構造構築

入力: 入力ファイル

出力: EXT 配列（extremity 集合）, ADJ リスト（adjacency 集合）

$N \leftarrow 1$ 行目から読み込む;

$EXT \leftarrow$ サイズ $2N$ の配列を初期化;

// ゲノム A の処理

2 行目をパースし，A の構成を取得;

foreach A **における隣接** $\{u, v\}$ **do**

  u, v に対応する EXT 内要素のポインタを相互に接続 (reality);

  $ADJ_NODE \leftarrow u, v$ を持つ新規 adjacency ノード;

  ADJ リストに追加;

// ゲノム B の処理

3 行目をパースし，B の構成を取得;

foreach B **における隣接** $\{x, y\}$ **do**

  ADJ リスト内の，x を含むノードと y を含むノードを探索;

  発見したノード間にエッジ（$left_to$ / $right_to$）を張る;

return EXT, ADJ;

Algorithm 2: 隣接グラフを用いた DCJ 距離の計算

入力: ゲノム A, ゲノム B

出力: DCJ 距離 d_{DCJ}

$N \leftarrow$ 遺伝子数;

$AG(A, B) \leftarrow A$ と B の隣接集合から隣接グラフを構築;

$C \leftarrow 0$, $I \leftarrow 0$;

foreach 未訪問の隣接ノード $v \in AG(A, B)$ **do**

  v から始まるコンポーネントをトラバースする;

  $L \leftarrow$ トラバースしたエッジ（隣接関係）の総数;

  if 探索が開始点に戻った（サイクルを発見） then

    $C \leftarrow C + 1$;

  else if 端（テロメア）に到達した（パスを発見） then

    if L **が奇数 then**

      $I \leftarrow I + 1$;

$d_{\mathrm{DCJ}} = N - (C + I/2)$;

return d_{DCJ};

3.3 正当性と計算量

本アルゴリズムの探索部分は，各エッジをちょうど一度ずつ走査するため，計算量は $O(N)$ となる．終了性については，各 adjacency に訪問済みフラグを設定し，一度の走査で全コンポーネントを網羅するため，有限時間内での停止が保証される．

第 4 章　アルゴリズム 2：操作手順の出力

[3] の理論を基に，実際の変異手順を逐次的に出力するアルゴリズムを実装した.

4.1　実装方針

本アルゴリズムでは，現在のゲノム状態を目標のゲノム状態へ貪欲法により近づけていく．具体的には，特定の接続が目標とする接続となっていない場合，DCJ 操作を適用してその接続を確立させていく.

Algorithm 3: DCJ 操作によるゲノムソート手順の生成

入力: 現在のゲノム A の端点集合 $exts$, 目標状態 $goals$

出力: 実行された DCJ 操作のログ

$n \leftarrow$ 遺伝子数, $c \leftarrow 0$;

// 第 1 段階：目標が通常の端点である接続を優先処理

for $i = 0$ **to** $2n - 1$ **do**

  $from \leftarrow exts[i]$;

  $to \leftarrow goals[i]$ に対応する B 上の目標端点;

  if to **が有効な端点** ***and*** $from$ **の現在の接続相手** $\neq tar$ **then**

    DCJ 操作を実行し，$from$ と to を隣接させる;

    DoubleCutJoin(A, src, tar);

    $c \leftarrow c + 1$;

// 第 2 段階：目標がテロメアである接続を処理

for $i = 0$ **to** $2n - 1$ **do**

  $from \leftarrow exts[i]$;

  $to \leftarrow goals[i]$ に対応する目標 (テロメア) ;

  if to **がテロメア** ***and*** $from$ **の現在の接続相手** $\neq tar$ **then**

    DCJ 操作を実行し，$from$ をテロメアにする;

    DoubleCutJoin(A, src, tar);

    $c \leftarrow c + 1$;

return c;

4.2 正当性

本アルゴリズムの正当性は，前述の補題 2.5（成分数の変化）および補題 2.6（奇数長パス数の変化）から直接的に導かれる．これらの補題により，1 回の DCJ 操作で $C + I/2$ が増加する最大量は 1 であることが示されている（補題 2.8）．

アルゴリズム 2 で採用している「目標の接続を順次確定させていく」という貪欲的な戦略は，隣接グラフの観点では「長さ 2 のサイクル（共通の隣接）」あるいは「長さ 1 のパス（共通のテロメア）」をグラフから分離独立させる操作に相当する．この操作は，確実に C を 1 増加させるか，あるいは I を 2 増加させることで，$C + I/2$ を常に 1 ずつ増加させる効果を持つ．したがって，各ステップごとに距離 d_{DCJ}（(2.3) 式）は確実に 1 ずつ減少し，最終的に $A = B$ となるまで最短回数でのソート手順を与えることが保証される．

第 5 章　評価

本研究で実装したプログラムの性能を検証するため，異なるスケールのデータセットを用いて実行時間およびメモリ使用量の計測を行った．

5.1　実験環境

実験に使用した計算機のスペックは以下の通りである．

- **機種名**: MacBook Air (15-inch, M3)
- **チップ**: Apple M3（8 コア: パフォーマンス 4，効率性 4）
- **メモリ**: 16 GB
- **OS**: macOS (Darwin Kernel Version 24.6.0)

5.2　データセット

遺伝子数 N を 10 から $100,000$ まで変化させ，ランダムに生成したゲノム対を入力として使用した．

5.3　実験結果

データの読み込みおよび構造体の構築を含めた全体の実行時間（表 5.1）と，構築済みのデータ構造に対するアルゴリズム部分のみの実行時間（表 5.2）をそれぞれ計測した．なお，メモリ使用量はピーク時の値を Byte から MB に換算して記載した．

表 5.1　全体実行時間（読み込み・構築含む）とメモリ使用量

遺伝子数 (N)	実行時間 (sec)	メモリ使用量 (MB)
10	0.001	0.003
100	0.001	0.03
1,000	0.007	0.26
10,000	0.179	2.55
100,000	20.827	25.55

表 5.2　アルゴリズム実行時間（読み込み・構築除く）

遺伝子数 (N)	実行時間 (sec)
10	0.001
100	0.001
1,000	0.002
10,000	0.005
100,000	0.051

5.4　考察

実験結果より，メモリ使用量に関しては遺伝子数 N に対して線形増加の傾向を示しており，$N = 100,000$ の規模でも約 26 MB 程度と，一般的な PC 環境において十分に動作する範囲に収まっていることが確認できる．

実行時間について，データの読み込みを含めた場合（表 5.1）は $N = 100,000$ で約 20 秒を要しているのに対し，アルゴリズム部分のみ（表 5.2）では約 0.05 秒で処理が完了している．この乖離の主な要因は，入力処理におけるテキストデータのパース（構文解析）にあると考えられる．本実装では入力の容易さと可読性を重視し，10 進数のテキスト形式を採用している．しかし，遺伝子数 N が増大すると，個々の遺伝子 ID を表す文字数（桁数）も $\log_{10} N$ のオーダーで増加するため，テキストデータの総容量およびパースにかかる時間は $O(N \log N)$ で増加する．

厳密な計算量理論の観点に立てば，$N \to \infty$ において数値を表現するビット長自体が増大するため，いかなる形式であれ完全な線形時間の達成は困難である．しかし，アルゴリズムの理論解析で一般に用いられる計算モデル（Word RAM Model）では，マシン語長に収まる整数の操作は定数時間 $O(1)$ とみなされる．この前提に近づけ，実用的な速度を得るためには，本来であればバイナリ形式の採用などによる入出力の最適化が為されるべきである．本研究では，データ作成の簡便さとデバッグ時の可読性を優先した結果，この入出力オーバーヘッドが残存することとなった．

一方，核心となる DCJ 距離計算および操作手順生成アルゴリズムそのものは，理論通り $O(N)$ の線形時間特性を示しており，大規模なデータセットに対しても実用的な時間内で解を導出可能であることが実証された．

参考文献

[1] A. Bergeron, J. Mixtacki, and J. Stoye, “A new linear time algorithm to compute the genomic distance via the double cut and join distance,” *Theoretical Computer Science*, 410 (2009), pp. 5300–5316.

[2] S. Hannenhalli and P. A. Pevzner, “Transforming men into mice (polynomial algorithm for genomic distance problem),” in: *Proceedings of FOCS 1995*, IEEE Press, 1995, pp. 581–592.

[3] A. Bergeron, J. Mixtacki, and J. Stoye, “A unifying view of genome rearrangements,” in: *Proceedings of WABI 2006*, LNBI, vol. 4175, Springer Verlag, 2006, pp. 163–173.

付録 A　ソースコードの公開

本研究で実装した線形時間アルゴリズムのソースコード，および検証に使用したデータセットは，以下の GitHub リポジトリにて公開している．本実装は C 言語により記述されており，ビルドには `make` コマンドを利用するが，通常のコンパイルも可能である．`README.md` を参照いただきたい．

- **リポジトリ名**: `dcj_bergeron`
- **URL**: `https://github.com/hiratannnnnn/dcj_bergeron`
- **最終アクセス日**: 2026 年 1 月 8 日

なお，本リポジトリには以下のファイルが含まれる．

1. `srcs/solve.c`: アルゴリズム 1（距離計算）およびアルゴリズム 2（操作手順出力）の主要実装
2. `srcs/double_cut_join.c`: DCJ 操作の基本機能の実装
3. `srcs/contexts.c`: 実行コンテキストおよびデータ構造の初期処理
4. `srcs/utils_*.c`: 入出力処理やデータ構造操作の補助関数群
5. `include/`: データ構造（extremity, adjacency 等）を定義したヘッダファイル群
6. `main.c`: プログラムのエントリーポイント
7. `Makefile`: ビルド用スクリプト
8. `README.md`: 実行方法および環境要件の解説